

Umbrella Notes — Module 2

Operating Systems | shared core for Coal India MT (Systems), IBPS SO IT Officer (SPL-XVI), DSSSB TGT CS (DOE)

How to read this module

This is **full teaching notes** — learn directly from it. Depth is **balanced across all three exams** (middle depth by default: enough for IBPS SO, DSSSB TGT, and general PSU/Govt CS-IT-Systems exams).

■ **Green box** = how PSU/Govt exams actually test the topic (a reference lens, not a limit).

↑ **Amber box** = extra GATE-core depth needed *only* if targeting Coal India MT (or GATE-style PSUs). Skip for IBPS/DSSSB.

■ **Purple box** = a coverage warning (something kept light, or where an exam needs more/less).

Small sub-topics are folded under umbrella headings but never dropped. Topic-wise Q&A with explanations comes as a separate set after your walkthrough.

0 What this module covers

An **operating system (OS)** is the software layer between the hardware and the user's programs. It has two faces: a **resource manager** (sharing CPU, memory, I/O among programs) and an **extended machine** (hiding messy hardware behind clean abstractions like files and processes). Almost every OS exam question maps to one of five jobs it does:

- **1. Process & thread management** — running programs, scheduling the CPU among them.
- **2. Synchronization & concurrency** — letting processes cooperate without corrupting shared data.
- **3. Deadlocks** — detecting/avoiding processes stuck waiting on each other.
- **4. Memory management** — allocating RAM, paging, virtual memory.
- **5. File & I/O / device management** — storing data, scheduling disk access.

■ Tested pattern & focus (PSU / Govt CS-IT-Systems)

OS is **high-frequency** in IBPS SO, DSSSB TGT and PSU CS papers. Most questions are direct concept MCQs: definitions, **scheduling algorithm** behaviour, **deadlock conditions**, **paging vs segmentation**, and **OS types**.

Guaranteed-favourite clusters: types of OS, process states, **CPU scheduling** (FCFS/SJF/RR), the four **Coffman deadlock conditions**, and **page replacement** (FIFO/LRU/Optimal).

1 OS Basics, Types & Structure

1.1 Functions & the kernel

The **kernel** is the core of the OS, always resident in memory, running in a privileged **kernel mode** (vs the restricted **user mode** that applications run in). Switching between them happens via a **system call** — the controlled doorway a program uses to ask the OS for a service (open a file, create a process, etc.).

1.2 Types of operating system

Type	Idea	Example / use
Batch	Jobs collected and run in batches, no interaction	Old mainframe payrolls
Multiprogramming	Several jobs in memory; CPU switches when one waits for I/O	Keeps CPU busy
Multitasking / Time-sharing	CPU time sliced among users/processes for interactivity	Desktop OS
Multiprocessing	Multiple CPUs/cores execute in parallel	Servers, modern PCs
Real-time (RTOS)	Guarantees response within a strict deadline	Airbags, medical, robotics
Distributed	Independent machines appear as one system	Clusters, cloud
Network	Lets machines share resources over a network	File/print servers

■ Hard vs soft real-time

Hard RTOS — missing a deadline is catastrophic (airbag). **Soft RTOS** — deadlines matter but occasional misses are tolerable (video streaming). A common one-line distinction MCQ.

↑ Go deeper — GATE-core (Coal India MT)

For Coal India / GATE, add OS **structures**: monolithic vs microkernel vs layered vs modular kernels, and the exact **system-call** mechanism (trap/interrupt, mode switch vs context switch distinction). IBPS/DSSSB need only the kernel/user-mode idea and the types table.

2 Processes, Threads & Scheduling

2.1 Process vs program vs thread

A **program** is passive code on disk. A **process** is a program in execution — an active entity with its own memory, registers, and state. A **thread** is a lightweight unit of execution *within* a process; threads of one process share its memory/code but have their own stack and registers. Multithreading gives concurrency cheaply.

The OS tracks each process in a **Process Control Block (PCB)** — holding its ID, state, program counter, registers, memory info, and open files. Switching the CPU from one process to another means saving one PCB and loading another: a **context switch** (pure overhead).

2.2 Process states

A process moves through states: **New** → **Ready** → **Running** → **(Waiting/Blocked)** → **Terminated**. It's **Ready** when it needs only the CPU, **Running** when on the CPU, and **Waiting** when blocked on I/O. The scheduler moves processes between Ready and Running.

2.3 CPU scheduling algorithms

The **scheduler** picks which ready process runs next; the **dispatcher** hands it the CPU. Key metrics: **CPU utilization, throughput, turnaround time, waiting time, response time**. **Preemptive** algorithms can interrupt a running process; **non-preemptive** cannot.

Algorithm	Rule	Preemptive?	Note
FCFS	First come, first served (queue order)	No	Simple; suffers convoy effect
SJF	Shortest job first	No	Optimal avg wait; needs burst prediction
SRTF	Shortest remaining time first	Yes	Preemptive SJF
Priority	Highest priority first	Either	Risk of starvation → fix with aging
Round Robin	Each gets a fixed time quantum, cyclically	Yes	Fair; good for time-sharing

■ Tested pattern & focus (PSU / Govt CS-IT-Systems)

Expect **numerical scheduling problems**: given arrival + burst times, compute average waiting/turnaround time for FCFS, SJF, or RR. Learn the Gantt-chart method.

Concept MCQs: which algorithm causes **starvation** (priority), **convoy effect** (FCFS), or needs a **time quantum** (RR); preemptive vs non-preemptive classification.

↑ Go deeper — GATE-core (Coal India MT)

GATE-side: multilevel & multilevel-feedback queues, exact formulas, and edge cases (RR with context-switch overhead, SJF optimality proof). Also solve for response time vs waiting time precisely.

3 Concurrency, Synchronization & Deadlocks

3.1 The critical-section problem

When processes share data, concurrent access causes a **race condition** — the result depends on timing. The block of code touching shared data is the **critical section (CS)**. A correct solution needs three properties: **mutual exclusion** (only one in the CS), **progress** (no needless blocking), and **bounded waiting** (no starvation).

3.2 Synchronization tools

- **Semaphore** — an integer with atomic **wait (P)** and **signal (V)** operations. **Binary** semaphore (0/1) = mutex; **counting** semaphore manages multiple identical resources.
- **Mutex lock** — a simple lock/unlock for mutual exclusion.
- **Monitor** — a higher-level construct bundling shared data with the procedures that guard it (used with condition variables).

■ Classic problems (know the name + idea)

Producer-Consumer (bounded buffer), **Readers-Writers**, and **Dining Philosophers** — standard illustrations of synchronization and deadlock. Exams ask which construct solves which.

3.3 Deadlocks

A **deadlock** is a set of processes each waiting forever for a resource held by another. It can occur only when **all four Coffman conditions** hold simultaneously:

- **Mutual exclusion** — a resource is held in non-shareable mode.
- **Hold and wait** — a process holds one resource while waiting for another.
- **No preemption** — resources can't be forcibly taken away.
- **Circular wait** — a closed chain of processes each waiting on the next.

Strategy	Idea
Prevention	Ensure at least one Coffman condition can never hold
Avoidance	Grant resources only if the system stays 'safe' — Banker's Algorithm
Detection & recovery	Allow deadlocks, detect via resource-allocation graph, then recover
Ignore (Ostrich)	Pretend it won't happen — used by many general-purpose OS

■ Tested pattern & focus (PSU / Govt CS-IT-Systems)

The **four Coffman conditions** are one of the single most-repeated OS MCQs — memorise them verbatim.

Know that **Banker's Algorithm** = deadlock *avoidance* (not prevention), and that breaking **circular wait** (e.g. resource ordering) is a common prevention method.

↑ Go deeper — GATE-core (Coal India MT)

GATE-side: run the **Banker's Algorithm** numerically (need/allocation/available matrices, safe sequence), and read **resource-allocation graphs** for deadlock (cycle = deadlock only with single-instance resources).

4 Memory Management & Virtual Memory

4.1 Why manage memory

Multiple processes must share limited RAM safely. The OS translates each process's **logical (virtual) addresses** into **physical addresses** (done by the **MMU** hardware), keeps processes isolated, and maximises utilization.

4.2 Allocation schemes

- **Contiguous allocation** — each process gets one continuous block. Suffers **external fragmentation** (free memory scattered in unusable gaps). Fixed-partition schemes suffer **internal fragmentation** (allocated block larger than needed).
- **Paging** — physical memory split into fixed **frames**, logical memory into equal **pages**; a **page table** maps pages to frames. Eliminates external fragmentation; small internal fragmentation in the last page.
- **Segmentation** — memory divided by logical units (code, stack, data) of variable size; suffers external fragmentation but matches program structure.

	Paging	Segmentation
Block size	Fixed (pages/frames)	Variable (logical segments)
Fragmentation	Internal	External
Viewpoint	Physical (OS)	Logical (programmer)
Address	Page no. + offset	Segment no. + offset

4.3 Virtual memory & paging in / out

Virtual memory lets a process run even if it doesn't fully fit in RAM, by keeping only needed pages in memory and the rest on disk (in the **swap** space). **Demand paging** loads a page only when referenced; a reference to a page not in RAM causes a **page fault**, and the OS fetches it from disk. Too much paging activity = **thrashing** (CPU spends more time swapping than working).

4.4 Page replacement algorithms

Algorithm	Replaces...	Note
FIFO	The oldest loaded page	Simple; suffers Belady's anomaly
LRU	The least-recently-used page	Good approximation of optimal
Optimal	The page not needed for the longest future time	Theoretical best; not implementable

■ Tested pattern & focus (PSU / Govt CS-IT-Systems)

Numericals: **count page faults** for a given reference string under FIFO/LRU/Optimal with N frames. Very common.

Concept hooks: **Belady's anomaly** (more frames → more faults) occurs in **FIFO**, not in LRU/Optimal. Distinguish **internal vs external fragmentation**. Know **page fault** and **thrashing** definitions.

↑ Go deeper — GATE-core (Coal India MT)

GATE-side: effective access time with TLB hit ratio, multi-level page tables, inverted page tables, and the paging-address arithmetic (page size, page-table size, offset bits).

5 File Systems & I/O / Disk

5.1 File management

A **file** is a named collection of related data; the **file system** organises files into **directories** and tracks where each file's blocks live on disk. File **allocation methods**: **contiguous** (fast, but fragmentation), **linked** (each block points to the next; no random access), and **indexed** (an index block lists all data blocks). Access control uses permissions (read/write/execute).

5.2 Disk scheduling

Because moving the disk head (**seek**) is slow, the OS orders pending disk requests to minimise head movement:

- **FCFS** — serve in arrival order (fair, not efficient).
- **SSTF** — shortest seek time first (nearest request next; risks starvation).
- **SCAN / Elevator** — head sweeps one direction servicing requests, then reverses.
- **C-SCAN** — like SCAN but only services in one direction, then jumps back (more uniform wait).
- **LOOK / C-LOOK** — SCAN/C-SCAN variants that reverse at the last request instead of the disk end.

■ Tested pattern & focus (PSU / Govt CS-IT-Systems)

Disk-scheduling numericals ask for **total head movement** given a request queue and starting head position — practise SCAN, SSTF, C-SCAN.

File allocation method comparisons (contiguous/linked/indexed) and their random-access ability are common concept MCQs.

■ Coverage note / warning

I/O hardware details (interrupt vs DMA, device drivers, spooling) are kept light here — low frequency for IBPS/DSSSB. For Coal India, revisit DMA and interrupt-driven I/O in **Module 5 (Computer Organization)**, where they fit more naturally.

Rapid recap — highest-yield facts

Area	Nail these
Basics	Kernel vs user mode; system call = doorway; OS types (batch/multiprog/time-sharing/RTOS/distributed)
Process	Program(passive) vs process(active) vs thread(lightweight); PCB; states New-Ready-Running-Waiting-Terminated; context switch = overhead
Scheduling	FCFS(convoy), SJF(optimal wait), RR(quantum, time-sharing), Priority(starvation→aging); preemptive vs not; Gantt numericals
Sync	Critical section needs mutual-exclusion+progress+bounded-wait; semaphore P/V; binary=mutex; monitors
Deadlock	4 Coffman: mutual excl, hold&wait;, no preempt, circular wait; Banker=avoidance; break circular wait to prevent
Memory	Paging(internal frag) vs segmentation(external); virtual memory; page fault; thrashing; FIFO(Belady)/LRU/Optimal
File/Disk	Allocation contiguous/linked/indexed; disk scheduling FCFS/SSTF/SCAN/C-SCAN; total head movement

✓ Before your walkthrough

That's the shared-core OS picture at balanced depth. Flag any page/section that felt shaky and I'll go deeper; then the **OS topic-wise Q&A set** (real PSU/Govt questions + explanations) follows for live practice.

Coal India target? Do the amber ↑ **GATE-core** boxes as a second pass — scheduling numericals, Banker's Algorithm, TLB/paging arithmetic, and page-fault problems are the extra depth OS needs.